CloudClean

AWS Resource Lifecycle & Cost Management Platform

Complete Technical Documentation

Live demo: http://13.233.99.134

Source: github.com/Amirtha655/CloudClean

July 2026

Contents

1. Overview

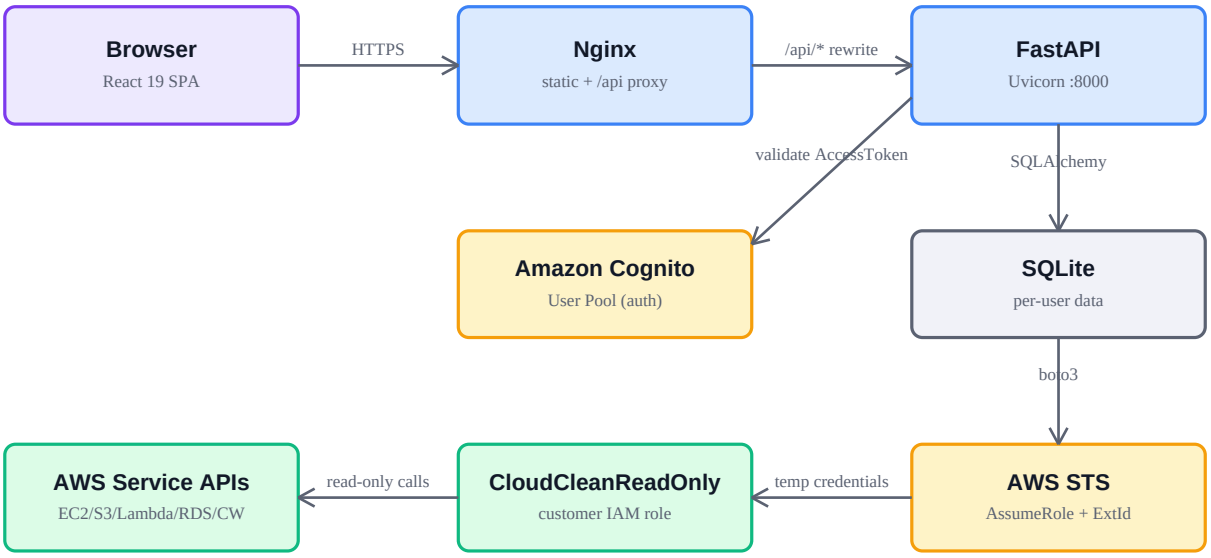
CloudClean is a full-stack web application that connects to a real AWS account, discovers its resources across multiple regions, maps the dependencies between them, and recommends safe cleanup actions with confidence scores. Instead of deleting resources blindly, users build a cleanup plan, see dependency warnings, preview the plan with a dry run that changes nothing, and only then execute — every executed run is logged and downloadable as a PDF report.

Everything in the product is real: authentication is a live Amazon Cognito user pool, scans make genuine boto3 API calls against AWS, dependency edges come from actual attachment data returned by EC2, idle-Lambda detection reads 30 days of real CloudWatch invocation metrics, and the trend charts are built from recorded scan snapshots — the application contains no synthetic or demo data.

What a user does, end to end

Step	What happens
1. Sign up / verify / log in	Real Cognito signup with an email verification code; login returns a Cognito AccessToken used as the API bearer token.
2. Connect an AWS account	User supplies an IAM role ARN + External ID for a read-only cross-account role they created in their own account.
3. Scan	Background job assumes the role via STS and inventories EC2, EBS, Elastic IPs, Security Groups, S3, Lambda, and RDS across 4 regions.
4. Review	Dashboard, Resource Explorer (filters/search), Dependency Graph, and confidence-scored Recommendations.
5. Plan & dry run	Select resources (individually, by template, or by environment), see dependency warnings and estimated savings; dry run changes nothing.
6. Execute & report	Execution marks resources deleted in CloudClean's own database (never calls destructive AWS APIs) and records a history entry with a downloadable PDF report.

2. System architecture



One EC2 instance hosts Nginx + FastAPI + SQLite; Cognito/STS/service APIs are AWS-managed.

The runtime is deliberately simple: a single EC2 instance runs Nginx and the FastAPI backend. Nginx serves the pre-built React bundle as static files and reverse-proxies every request under `/api/` to Uvicorn on localhost, stripping the prefix. Because frontend and API share one origin, no CORS configuration is needed in production.

The FastAPI process talks to three AWS control planes: Cognito to validate bearer tokens on every request, STS to assume the customer's read-only role at scan time, and the individual service APIs (EC2, S3, Lambda, RDS, CloudWatch) through the temporary credentials STS returns. Application state lives in SQLite through SQLAlchemy ORM models that port unchanged to PostgreSQL.

Request flow for a scan

POST /accounts/{id}/scan → ownership check → status set to *scanning* → FastAPI BackgroundTask opens its own database session → STS AssumeRole with the stored External ID → per-region boto3 inventory calls → previous snapshot rows replaced atomically → recommendations regenerated → a ScanSnapshot row recorded for the trend charts → status back to *connected*. The HTTP response returns in ~300 ms; the frontend polls every 3 seconds while any account is scanning and refreshes all data queries when it finishes.

3. Tech stack and why each piece was chosen

Frontend

Technology	Why it was chosen
React 19 + TypeScript	Component model fits a many-screen dashboard app; TypeScript catches API-shape mistakes at compile time — the backend emits camelCase JSON matching the frontend types 1:1.
Vite	Sub-second dev startup and HMR; the dev server proxies /api to the backend with the same rewrite Nginx uses in production, so dev and prod behave identically.
Tailwind CSS 4	The whole theme is design tokens (@theme variables) — the dark→light redesign was mostly a token swap because components reference semantic names like bg-surface, not raw colors.
TanStack Query	Server state done right: caching, request deduplication, conditional polling (accounts refetch every 3 s only while a scan is running), and targeted cache invalidation after mutations like scan/disconnect/execute.
React Router 7	Client-side routing for the 11-screen app shell; Nginx falls back to index.html so deep links work.
Recharts	Declarative React charting for the dashboard pie/bar/area charts — no imperative D3 code to maintain.

Backend

Technology	Why it was chosen
FastAPI	Typed request/response models via Pydantic give a self-documenting API; dependency injection cleanly expresses auth (get_current_user) and DB sessions; BackgroundTasks covers async scanning without adding a queue.
Pydantic v2	One CamelModel base class converts snake_case Python to camelCase JSON automatically, keeping both codebases idiomatic.
SQLAlchemy 2	Typed ORM models (Mapped[...]); SQLite locally and in the demo deployment, identical models on PostgreSQL/RDS when needed.
Boto3	The official AWS SDK — STS role assumption, per-service inventory calls, CloudWatch metrics, and Cognito administration all through one library.
ReportLab	Generates the per-cleanup PDF reports (and this document) in pure Python — no headless browser needed.
Uvicorn	Standard ASGI server; runs as a systemd service bound to localhost, fronted by Nginx.

AWS services

Service	Role in CloudClean
Cognito	Managed auth: signup, email verification codes, password policy, token issuance. The backend never stores passwords.
STS	Cross-account access with temporary credentials — customers grant a read-only role instead of handing over access keys.
EC2 / S3 / Lambda / RDS APIs	The inventory sources: instances, volumes, addresses, security groups, buckets, functions, databases.
CloudWatch	GetMetricStatistics over 30 days of Lambda Invocations — the real idle-function signal.

Service	Role in CloudClean
Systems Manager	Run Command deploys to the instance with zero SSH; Parameter Store (SecureString) holds the Cognito client secret.
EC2 (hosting)	One free-tier t3.micro on Amazon Linux 2023 hosts the whole app behind Nginx.
IAM	Least-privilege everywhere: the instance role can only describe/list the scanned services, read the deploy bucket, and decrypt its own parameters.

4. How the scanner works

A scan is a full re-inventory: the previous snapshot for the account is deleted and replaced, so the database always reflects the latest real state. The scanner modules live in `backend/app/aws/scanners/` — one small file per service.

Scanner	boto3 calls	What it extracts
ec2.py	describe_instances, describe_volumes, describe_addresses, describe_security_groups	Instances (state, type, tags, stopped-since parsed from StateTransitionReason), volumes (size, attachment), Elastic IPs (association), security groups — plus dependency edges from real attachment data.
s3.py	list_buckets, get_bucket_location, list_objects_v2, get_bucket_tagging	All buckets (S3 is global); emptiness via KeyCount==0; approximate size from the first 1,000 objects.
lambda_.py	list_functions (paginated), list_tags, cloudwatch get_metric_statistics	Every function, with a 30-day Invocations sum — zero invocations marks it unused. Unknown metric access never falsely flags a function.
rds.py	describe_db_instances (paginated)	Database instances with class, status, and tags.

The orchestrator (`scanner.py`) loops over the account's four configured regions (us-east-1, us-west-2, eu-west-1, ap-south-1), wraps each service scan in its own exception handler so one failing service never aborts the whole scan, then writes resources, dependency edges, risk scores, recommendations, and the trend snapshot in one transaction. A full scan of a real account with ~25 Lambda functions takes 30–90 seconds, dominated by one CloudWatch call per function.

5. The dependency graph engine

Dependency edges are never inferred or guessed — they are read from the attachment fields AWS itself returns: an instance's BlockDeviceMappings yield EC2→EBS edges, its SecurityGroups yield EC2→SG edges, and an address's InstanceId yields EC2→EIP edges. The graph endpoint walks the stored edges around a chosen resource and reports every affected neighbour.

Risk is derived from real fan-in: a resource with 2 or more dependents is rated HIGH, exactly one dependent is MEDIUM, none is LOW. The dashboard risk score aggregates these with weights (low=1, medium=4, high=9) normalised to 0–100. The cleanup planner reuses the same edges to warn when a selected resource is still used by something outside the selection — e.g. "ebs-vol-X is used by ec2-instance-Y" — before anything is executed.

6. The recommendation engine

Deliberately rule-based over real signals rather than ML — every recommendation is explainable and traceable to an observable fact about the resource. These are the actual rules and constants from `scanner.py`:

Signal (from real AWS data)	Action	Confidence
EC2 instance state == stopped (days parsed from StateTransitionReason)	Terminate	min(60 + days stopped, 98)%
S3 bucket KeyCount == 0	Delete	90%
Lambda: 30-day CloudWatch Invocations sum == 0	Delete	92%
Elastic IP with no InstanceId/AssociationId	Release	95%
EBS volume with no Attachments	Delete	88%

Each recommendation carries the resource's estimated monthly cost as its savings figure and links directly into the cleanup planner. The one dynamic element is the EC2 confidence, which grows with how long the instance has been stopped — an instance stopped 47 days scores 98%, one stopped 8 days scores 68%.

7. Cleanup planner, dry run, and execution

Users assemble a selection three ways: individually in the Resource Explorer, in bulk with the environment quick-select (keep production / delete development), or via templates — stored filter presets (e.g. "Everything Except Production" applies excludeEnvironment=production in the Explorer).

Creating a plan computes the per-service breakdown, total estimated monthly savings, an estimated duration, and dependency warnings. **Dry run** returns the full outcome without touching any row. **Execute** marks the selected resources deleted in CloudClean's database and writes a CleanupHistory row (who, when, counts, savings, resource names) — by design it never calls a destructive AWS API; the instance role does not even hold delete permissions, so the safety is enforced by IAM, not just code. Each history row has a downloadable ReportLab PDF report.

8. Authentication and multi-tenancy

Signup calls Cognito SignUp (with a computed SECRET_HASH), which emails a 6-digit verification code; confirm calls ConfirmSignUp; login uses USER_PASSWORD_AUTH and returns the **AccessToken** as the app's bearer token. On every authenticated request the backend calls Cognito GetUser with that token — cryptographic validation is delegated to AWS, so no JWKS handling or JWT library exists in the codebase. The user row (keyed by the immutable Cognito sub) is upserted on first sight.

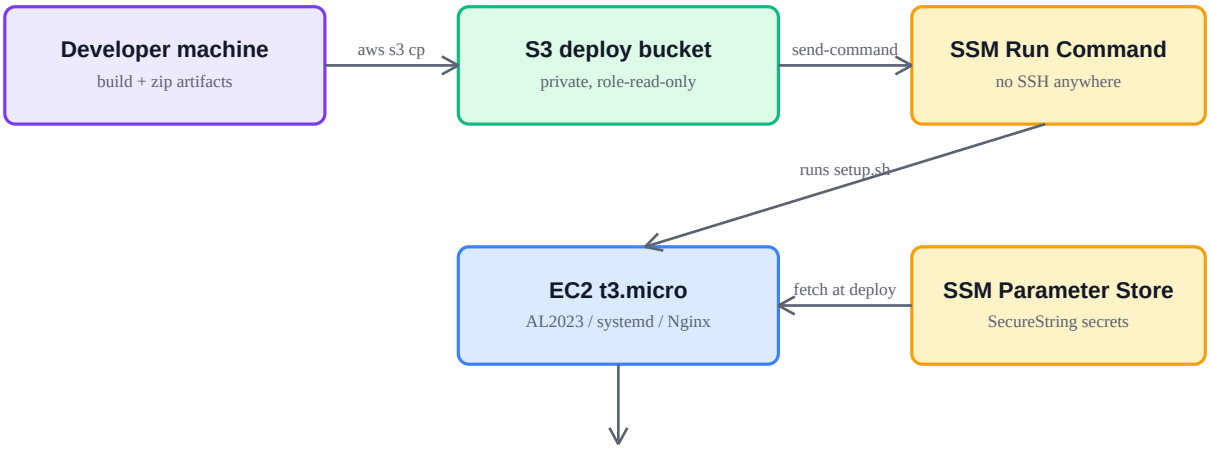
Multi-tenancy is enforced at the query layer: AwsAccount carries user_id, and every data endpoint (accounts, resources, dashboard, recommendations, dependencies, cost, planner) filters through a join on AwsAccount.user_id == current_user.id. A brand-new user sees an empty dashboard until they connect and scan their own account. The frontend clears the token and redirects to /login on any 401.

9. Cost estimation and scan-history trends

Monthly cost figures are transparent estimates from a static pricing table (per EC2 instance type, per RDS class, per-GB EBS/S3, flat Lambda estimate) rather than AWS Cost Explorer, because Cost Explorer must be manually enabled per account and takes ~24 h to activate. The README and UI label them as estimates.

Trend charts contain no synthetic points: every completed scan records a ScanSnapshot (timestamp, total resources, total estimated cost), and the dashboard/cost trends plot exactly those snapshots — one point per day per account (latest scan wins), summed across the user's accounts. Until a second scan exists the UI says so honestly instead of drawing a fake curve.

10. Deployment architecture



Instance pulls artifacts from S3 and secrets from Parameter Store using its own IAM role.

Deployment never opens SSH: artifacts (frontend dist, backend app + requirements) are zipped and uploaded to a private S3 bucket, then an SSM Run Command executes `deploy/setup.sh` on the instance — installing Python 3.12 (AL2023's default 3.9 cannot parse the codebase's modern type hints), building the venv, fetching configuration from SSM Parameter Store (the Cognito client secret is a SecureString decrypted only by the instance role), and writing the systemd unit and Nginx site. The script is idempotent — re-running it is the whole redeploy story.

Component	Detail
Instance	EC2 t3.micro (free tier), Amazon Linux 2023, ap-south-1
Web server	Nginx: static frontend at /, /api/* proxied to 127.0.0.1:8000 with prefix rewrite
App server	Uvicorn under systemd (Restart=always), runs as ec2-user
Secrets	SSM Parameter Store /cloudclean/prod/* — never in the deploy package or repo
Instance IAM role	Describe/List on scanned services + CloudWatch read + S3 deploy-bucket read + scoped SSM/KMS read + AmazonSSMManagedInstanceCore
Network	Security group allows inbound 80/443 only; no SSH port at all

11. Security model

Concern	How it is handled
Customer credentials	None stored. Customers create a role in their own account trusting CloudClean's platform role, protected by an External ID (confused-deputy defence). STS issues short-lived credentials per scan.
Blast radius of a scan	The customer role's policy is describe/list-only; CloudClean's own instance role also has zero destructive permissions — 'cleanup' is bookkeeping in CloudClean's database.
Cross-account fallback	If AssumeRole fails, local platform credentials are used only when the account ID equals the platform's own demo account; any other account fails with a clear error instead of silently scanning the wrong account.
Token handling	Cognito AccessToken as bearer; validated against Cognito on every request; cleared client-side on 401; login/signup/verify are the only unauthenticated data endpoints.
Tenant isolation	All queries join-filter on the owning user id; object-level checks (e.g. _get_owned_account) 404 on other users' resources.

Concern	How it is handled
Secrets in deployment	SecureString parameters fetched at deploy time by the instance role; deploy scripts avoid shell tracing around secret material; .env files are git-ignored and excluded from artifacts.

12. Data model

Table	Purpose	Key fields
users	One row per Cognito identity	id = Cognito sub, email, name
aws_accounts	A connected AWS account owned by a user	user_id FK, aws_account_id, role_arn, external_id, status (pending/scanning/connected/error), regions, last_scan_at
resources	One row per discovered AWS resource (replaced each scan)	account_id FK, service, type, name, arn, region, state, tags, monthly_cost, last_used_days, environment, risk_score, unused, deleted
dependency_edges	Real attachment relationships	source_id → target_id (both resource FKs)
recommendations	Engine output per unused resource	resource_id FK, reason, action, confidence, estimated_savings
scan_snapshots	Trend history — one per completed scan	account_id FK, taken_at, total_resources, total_cost
cleanup_plans	A prepared selection with warnings	resource_ids, estimated_savings, estimated_duration_minutes, warnings
cleanup_history	Every executed run	user, started/finished, status, resources_deleted/failed, savings, resource_names
cleanup_templates	Reusable Explorer filter presets	name, description, filters (JSON)

13. API reference

Method + path	Auth	Description
POST /auth/signup	—	Cognito signup; sends verification email
POST /auth/verify	—	Confirms the emailed code
POST /auth/login	—	Returns Cognito AccessToken
GET /auth/me	Bearer	Current user's name/email
GET/POST /accounts	Bearer	List / connect accounts (scoped to user)
POST /accounts/{id}/validate	Bearer	One STS call to test the trust role
POST /accounts/{id}/scan	Bearer	Starts the async background scan
DELETE /accounts/{id}	Bearer	Disconnect; cascades resources/edges/recommendations
GET /resources	Bearer	Filterable inventory (region/service/environment/risk/unused/excludeEnvironment/search)
GET /dependencies/{resourceId}	Bearer	Dependency graph + affected list + risk
GET /recommendations	Bearer	Confidence-sorted recommendations
POST /planner/plan	Bearer	Build a plan with warnings
POST /planner/plan/{id}/execute?dry_run=	Bearer	Dry run (no changes) or execute
GET /dashboard/summary	Bearer	Totals, breakdowns, real trend
GET /cost/analytics	Bearer	Cost views + real trend + top expensive
GET /history	Bearer	Cleanup run log

Method + path	Auth	Description
GET /reports/cleanup/{id}.pdf	—	PDF report for a run
GET /templates	Bearer	Filter presets
GET /admin/stats	Bearer	Platform-wide counts

14. Known limitations and future work

Limitation	Honest detail / natural next step
Service coverage	7 services scanned (EC2, EBS, EIP, SG, S3, Lambda, RDS) in 4 fixed regions; the per-service scanner modules make adding more mechanical. Region list should become user-selectable.
Cost accuracy	Static estimate table, not Cost Explorer; integrating the Cost Explorer API is the upgrade path once enabled on an account.
Execution semantics	Execution is intentionally non-destructive (database-only). Real deletion would need explicit delete permissions on the customer role plus dependency-ordered deletion and rollback handling.
Shared data	Cleanup history and templates are global rather than per-user; accounts and resources are fully isolated.
Transport	Live demo is plain HTTP on a public IP — a domain + TLS (e.g. Caddy or ACM behind an ALB) is the obvious hardening step.
Persistence	SQLite on the instance disk; the SQLAlchemy models are Postgres-ready when durability matters.
Testing	No automated suite yet; pytest + moto (mocked AWS) fits the scanner design well.

Generated from the CloudClean codebase — every rule, constant, permission, and flow described above is taken directly from the source at github.com/Amirtha655/CloudClean.